НАЦИОНАЛЬНЫЙ ИССЛЕДОВАТЕЛЬСКИЙ УНИВЕРСИТЕТ ИТМО

Факультет программной инженерии и компьютерных технологий

Тестирование программного обеспечения

Лабораторная работа № 1

Выполнили студенты

Набокова Алиса,

Шнейдерис Герардас

Группа № P3320

Преподаватель:

Демурчян Владимир Варламович

г. Санкт-Петербург

2026

Лабораторная работа № 1 1
Задание: **3**
Задание 1: **4**
Задание 2: **6**
Задание 3: **7**
Вывод: **10**

Вариант: 409910

Задание:

1. Функция *cos(x)*
2. Программный модуль для поразрядной сортировки массива
3. Описание предметной области:

   После того, как толпа вновь разразилась ликующими криками, Артур обнаружил, что он скользит по воздуху к одному из величественных окон во втором этаже здания, перед которым стоял помост, с которого оратор обращался к народу.

Задание 1:

Тестируемая функция - **cos(x)**

Область определения - все вещественные числа. Анализ эквивалентности позволил выделить следующие зоны эквивалентности:

- зоны монотонного возрастания функции — $x \in [-\pi + 2\pi k; 0 + 2\pi k] \forall k \in \mathbb{Z}$
- зоны монотонного убывания функции — $x \in [0 + 2\pi k; \pi + 2\pi k] \forall k \in \mathbb{Z}$
- локальные максимумы — $x \in \{2\pi k\} \forall k \in \mathbb{Z}$
- локальные минимумы — $x \in \{\pi + 2\pi k\} \forall k \in \mathbb{Z}$
- точки перемены знака (с - на +) — $x \in \{-\frac{\pi}{2} + 2\pi k\} \forall k \in \mathbb{Z}$
- точки перемены знака (с + на -) — $x \in \{\frac{\pi}{2} + 2\pi k\} \forall k \in \mathbb{Z}$
- точки с большим абсолютным значением

помимо особенных точек нужно протестировать и точки из их окрестностей. Нужно убедиться, что функция в окрестностях экстремумов не выходит из множества значений и в окрестностях корней у функции верный знак. Помимо все прочего нужно протестировать несколько точек с разницей в $2\pi k$, чтоб убедиться, что реализация сохраняет периодичность функции.

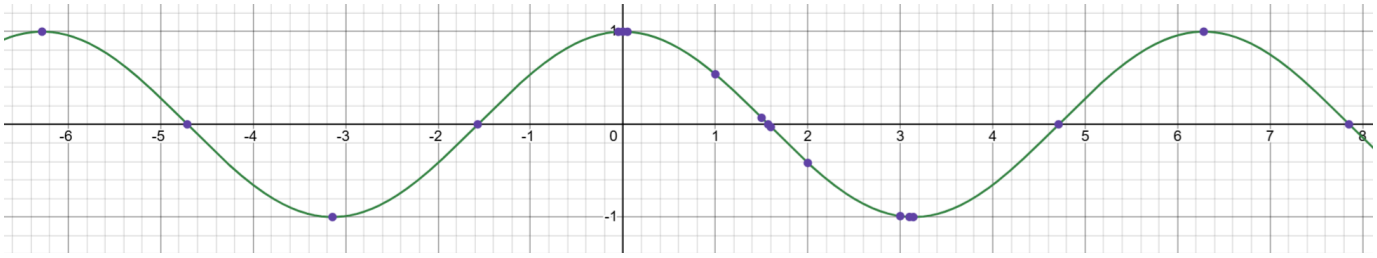


Рисунок 1 — [график](#) тестируемой функции с отмеченными точками в Desmos

В таблице 1 один приведены значения аргумента и функции в выбранных для тестирования точках (расчет значений был произведен с помощью **Desmos**).

x	cos(x)	x	cos(x)	x	cos(x)	x	cos(x)
-2π	1	-3π/2	0	-0.05	0.9988	1	0.5403
-π	-1	-π/2	0	0.05	0.9988	2	-0.4161
0	1	π/2	0	1.5	0.0707	3	-0.9899
π	-1	3π/2	0	1.6	-0.0292	-20	0.4081
2π	1	5π/2	0	3.1	-0.9991	1488	0.4403

Таблица 1 — значения аргумента и тестируемой функции в некоторых точках

Руководствуясь этими знаниями мы разработали нашу функцию и тесты к ней. Исходный код разработанной программы доступен по ссылке. Затем было проведено модульное тестирование функции на значениях из таблицы.

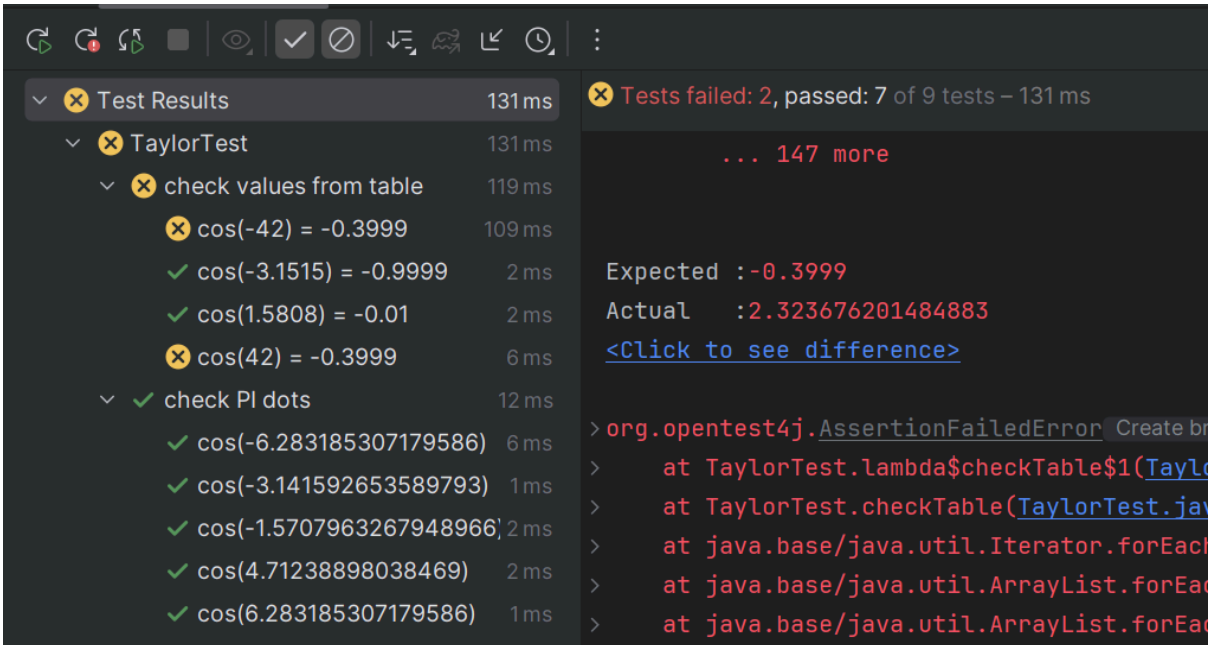


Рисунок 2 — результаты тестирования функции

Написанная функция отработала корректно на всех точек, кроме двух с большим абсолютным значением. Это логично ведь значение ряда совпадает со значениями функции только в некоторой окрестности нуля. Чтоб исправить эту ошибку можно перед вычислением использовать периодичность косинуса и приводить аргумент к промежутку [-π; π].

Для этого мы добавили в программу одну строку:

```
x = Math.abs(x) % (2 * Math.PI);
```

И все тесты были пройдены✅

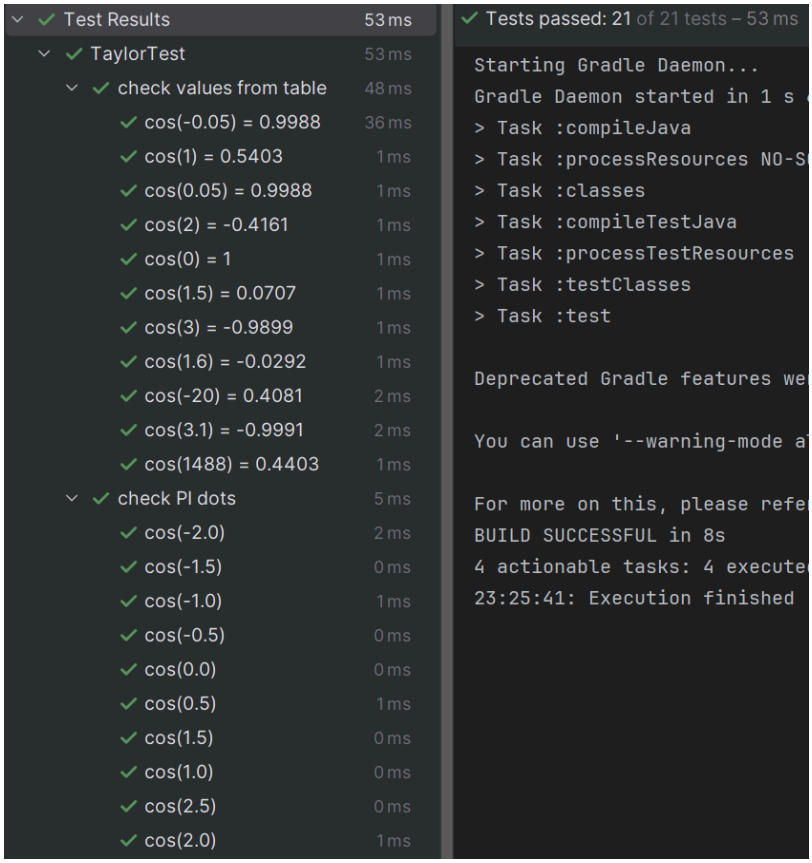


Рисунок 3 — результаты повторного тестирования функции

Также мы проверили значение ***test coverage*** с помощью **Intellij IDEA**

© Taylor	100% (1/1)	100% (1/1)	100% (7/7)	100% (2/2)

Рисунок 4 — процент тестового покрытия тестируемого класса

Задание 2:

Тестируемый алгоритм - **Radix Sort**

Этот алгоритм завязан на цифрах чисел в массиве, поэтому мы должны добавить в тесты числа с разным количеством разрядов в том числе однозначные и числа из большого числа знаков (например больше восьми). Отдельно должны быть протестированы случаи полностью и частично отрицательным массивом ведь алгоритм по умолчанию не обрабатывает такие случаи и это задача реализации. Более того должны быть протестированы тривиальные и некорректные случаи (в частности: неопределенный массив, массивы из 0, 1 и 2 элементов). Тестировать случаи с некорректными или неопределенными элементами массива не нужно, в силу типизации в Java. Код реализации и тестов доступен по той же ссылке.

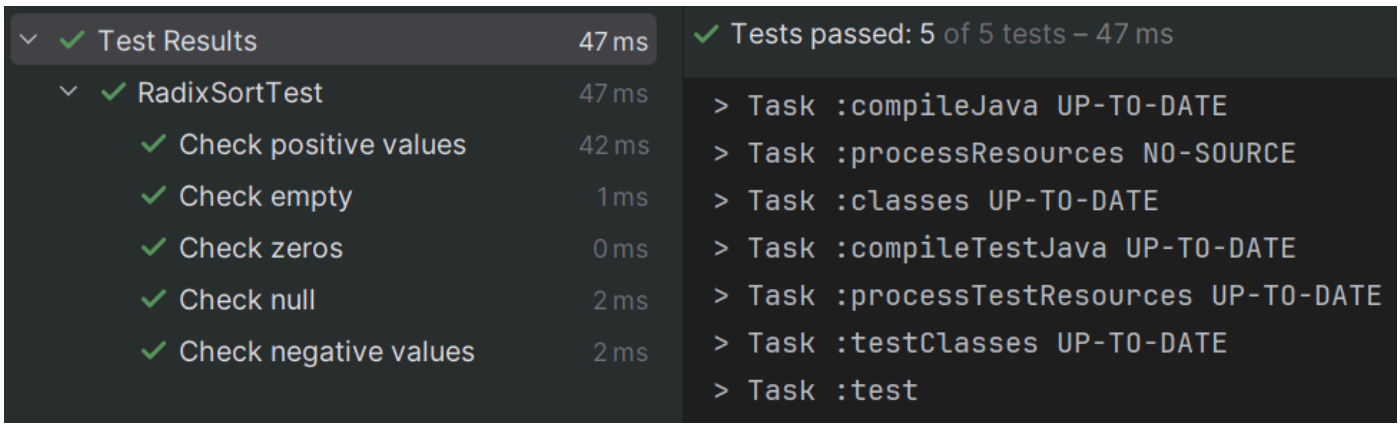


Рисунок 5 — результаты тестирования функции сортировки

Element	Class, %	Method, %	Line, %	Branch, %
ru.tpo.pikt.itmo	33% (1/3)	50% (2/4)	80% (33/41)	91% (31/34)
Main	0% (0/1)	0% (0/1)	0% (0/1)	100% (0/0)
RadixSort	100% (1/1)	100% (2/2)	100% (33/33)	96% (31/32)

Рисунок 6 — процент тестового покрытия тестируемого класса

Задание 3:

Мы разрабатывали доменную модель по тексту:

После того, как толпа вновь разразилась ликующими криками, Артур обнаружил, что он скользит по воздуху к одному из величественных окон во втором этаже здания, перед которым стоял помост, с которого оратор обращался к народу.

Модель представлена на рисунке 7 также был написан тестовый сценарий.

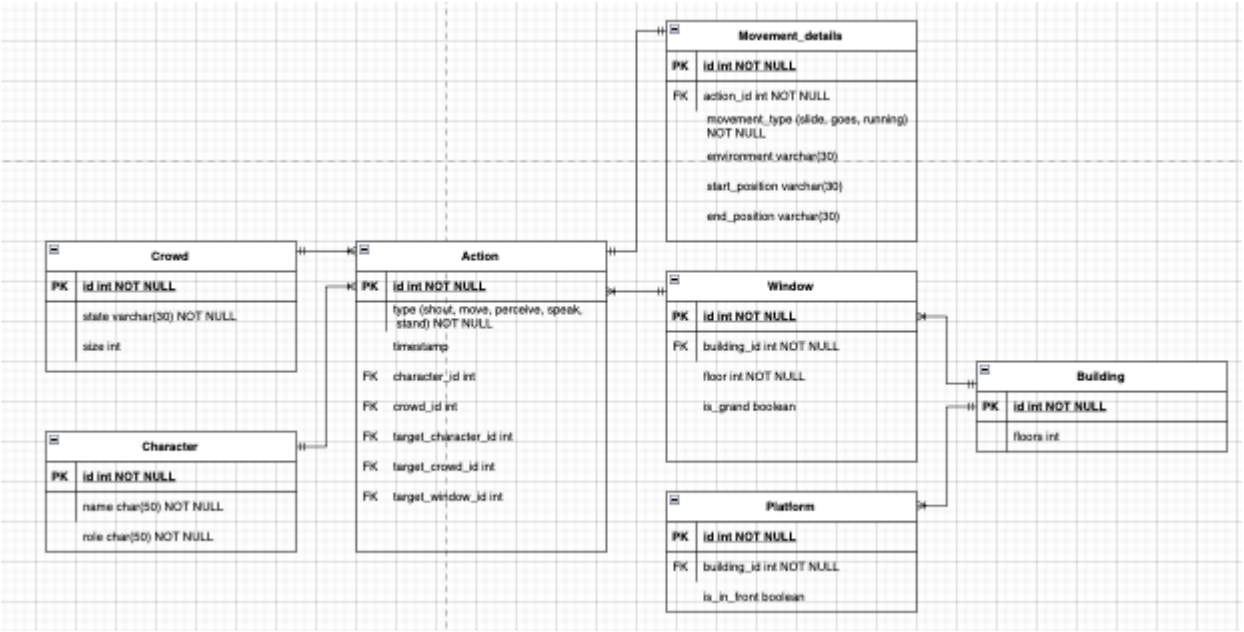


Рисунок 7 — диаграмма доменной модели

Диаграмма доменной модели доступна по ссылке.

Таблица 1. Тестовый сценарий полной истории

#	Начальное состояние	Ввод	Действие системы	Итоговое состояние
1	Система не содержит объектов	Создание персонажа Артур	Создаётся объект Character(id=1, name="Артур", role=MAIN)	В системе существует персонаж Артур
2	В системе есть Артур	Создание персонажа Оратор	Создается объект Character(id=2, name="Оратор", role=MINOR)	В системе существует персонаж Оратор
3	Персонажи существуют	Создание толпы	Создается объект Crowd(id=1, state="спокойная", size=500)	В системе существует толпа
4	Толпа существует	Создание действия SHOUT	Создается действие Action(type=SHOUT) с участием толпы	Толпа выполняет действие крика
5	Артур и толпа существуют	Создание действия PERCEIVE	Создается действие Action(type=PERCEIVE) для персонажа Артур	Артур воспринимает происходящее
6	Артур существует	Создание действия MOVE	Создается действие перемещения Action(type=MOVE)	Артур начинает перемещение
7	Существует действие перемещения	Добавление деталей перемещения	Создается MovementDetails(type=SLIDES, environment="воздух", end="окно во втором этаже здания")	Артур скользит по воздуху к окну
8	В системе нет здания	Создание здания	Создается объект Building(id=1, floors=2)	Существует здание с двумя

				этажами
9	Существует здание	Создание окна	Создается объект Window(id=1, floor=2, is_grand=true)	В здании существует величественное окно на втором этаже
10	Существует здание	Создание помоста	Создается объект Platform(id=1, is_in_front=true)	Перед зданием существует помост
11	Существует оратор и толпа	Создание действия SPEAK	Создается действие Action(type=SPEAK) с целью толпы	Оратор обращается к толпе

Вывод реализации сценария показан на рисунке 8.

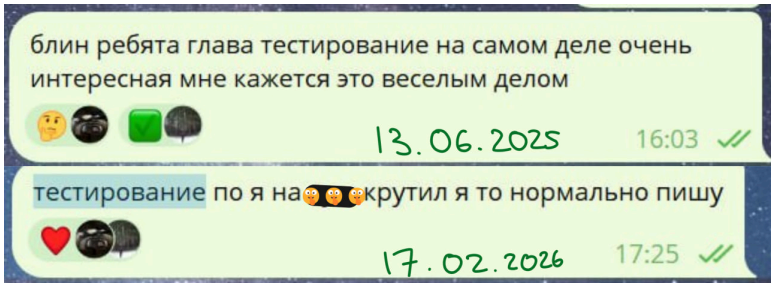
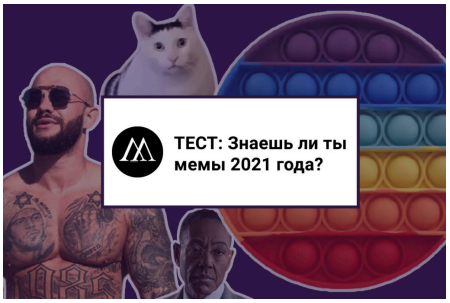
```
Crowd of 500 (ликующая) do SHOUT
Артур (MAIN) do PERCEIVE
Артур (MAIN) do MOVE
 details: SLIDES from - to окно во втором этаже здания in the воздух
Оратор (MINOR) do SPEAK to Crowd of 500 (ликующая)

Информация о действующих персонажах и объектах в истории:
Артур (MAIN)
Оратор (MINOR)
Crowd of 500 (ликующая)
Building №1 (2 floors)
  Window №1 on floor 2 (grand)
  Platform №1 in front
```

Рисунок 8 — Вывод сценария

По модели и сценарию мы написали код и тесты к нему получив покрытие показанное на рисунке 9.

entities	100% (12/...	94% (35/37)	94% (108/114)	87% (56/64)
actions	100% (4/4)	100% (12/12)	97% (41/42)	95% (21/22)
Action	100% (1/1)	100% (6/6)	100% (23/23)	100% (18/18)
ActionType	100% (1/1)	100% (2/2)	100% (5/5)	100% (0/0)
MovementDetails	100% (1/1)	100% (2/2)	90% (10/11)	75% (3/4)
MovementType	100% (1/1)	100% (2/2)	100% (3/3)	100% (0/0)
actors	100% (4/4)	91% (11/12)	96% (27/28)	100% (10/10)
Actor	100% (1/1)	50% (1/2)	80% (4/5)	100% (2/2)
Character	100% (1/1)	100% (5/5)	100% (13/13)	100% (6/6)
Crowd	100% (1/1)	100% (3/3)	100% (7/7)	100% (2/2)
Role	100% (1/1)	100% (2/2)	100% (3/3)	100% (0/0)
buildings	100% (4/4)	92% (12/13)	90% (40/44)	78% (25/32)
Building	100% (1/1)	100% (5/5)	95% (21/22)	91% (11/12)
BuildingCompone	100% (1/1)	75% (3/4)	76% (10/13)	66% (8/12)
Platform	100% (1/1)	100% (2/2)	100% (3/3)	50% (1/2)
Window	100% (1/1)	100% (2/2)	100% (6/6)	83% (5/6)

Рисунок 9 — процент тестового покрытия тестируемых классов

Вывод:

мы устали